

SAVITRIBAI PHULE PUNE UNIVERSITY



**A PROJECT STAGE II REPORT
ON**

**A SECURE AND SCALABLE FRAMEWORK FOR STUDENT
RECRUITMENT USING BLOCKCHAIN AND AI**

SUBMITTED TOWARDS THE PARTIAL FULFILLMENT OF THE REQUIREMENTS OF

BACHELOR OF ENGINEERING

BY

Kush Agrawal	Exam No: B401220406
Varad Dongarkar	Exam No: B401220414
Chinmay Kakade	Exam No: B401220424
Aastha Shinde	Exam No: B401220462

Under The Guidance of

Prof. Rupali Shishupal



DEPARTMENT OF INFORMATION TECHNOLOGY

Pimpri Chinchwad College of Engineering And Research, Ravet

A.Y. 2025–2026



Pimpri Chinchwad College Of Engineering And Research
DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the Project Entitled

**A SECURE AND SCALABLE FRAMEWORK FOR STUDENT
RECRUITMENT USING BLOCKCHAIN AND AI**

Submitted by

Kush Agrawal	Exam No: B401220406
Varad Dongarkar	Exam No: B401220414
Chinmay Kakade	Exam No: B401220424
Aastha Shinde	Exam No: B401220462

This is a bonafide work carried out by Students under the supervision of **Prof. Rupali Shishupal** and it is submitted towards the partial fulfillment of the requirement of **Bachelor of Engineering** (Information Technology) Project.

Prof. Rupali Shishupal

Internal Guide

Dr. Santoshkumar V. Chobe

H.O.D.

Dr. Pallavi G. Patil

Project Co-ordinator

Prof. Dr. H. U. Tiwari

Director

External Signature

Acknowledgments

We take great pleasure in presenting the project report on ‘**A Secure and Scalable Framework for Student Recruitment Using Blockchain and AI**’.

We would like to take this opportunity to thank our internal guide **Prof. Rupali Shishupal** for providing us with all the help and guidance needed. We are truly grateful for their kind support and valuable suggestions, which were extremely helpful throughout the project. We would also like to express our sincere thanks to our Project Coordinator **Dr.Pallavi G. Patil** for her guidance and continuous support.

We are also grateful to **Dr.Santoshkumar V. Chobe**, Head of Information Technology Department, PCCOER, Ravet for his indispensable support and valuable suggestions.

We are heartily thankful to **Prof. Dr. Harish U. Tiwari**, Director, PCCOER,Ravet, for providing all the support, guidance, research environment and constant motivation to improve our research profile.

Kush Agrawal

Varad Dongarkar

Chinmay Kakade

Aastha Shinde

(B.E.I.T.)

Abstract

Modern academic placement services and institutional degree verification procedures are structurally constrained by two systemic problems: the rise of falsified student qualifications and the significant cognitive friction associated with manually evaluating massive resume submission streams. To overcome these challenges, this paper presents the design and experimental validation of the Student Placement and Coordination Platform, implemented via an enterprise-grade, decentralized system known as PlacementEdge. To secure document provenance, the platform integrates a permissioned Hyperledger Fabric 2.5 blockchain ledger that records non-repudiable academic credential hashes. High-frequency, sub-100ms queries are handled through a read-optimized Supabase PostgreSQL layer using a synchronized CQRS architecture. Concurrently, the platform leverages Google Gemini 2.5 Flash via LangChain and LangGraph to automate resume extraction, avoiding unconstrained models that introduce ranking drift and demographic biases. Screened candidates are ranked using a deterministic bi-axial evaluation formula that separates job-relevant criteria (Criterion Validity) from demographic signals. To protect the platform's token stream from application-level exploits, RAPIDS (Resume Attack Prompt Injection Detection at Scale) and LocalAlign are deployed, reducing Indirect Prompt Injection (IDPI) success rates to below 10%. The PlacementEdge framework successfully integrates permissioned blockchain technology with an advanced AI screening module to mitigate credential fraud and accelerate recruitment processing.

INDEX

1	Introduction	1
1.1	Overview	2
1.2	Background and Motivation	2
1.3	Problem Statement	2
1.4	Proposed Solution and Contributions	3
1.5	Need and Relevance	4
1.6	Aim and Objectives	4
1.7	Project Scope and Limitations	5
1.7.1	Project Scope	5
1.7.2	Limitations	6
1.8	Organisation of the Report	6
2	Literature Survey	7
2.1	Overview of Existing Research	8
2.1.1	Decentralized Academic Credential Verification	8
2.1.2	AI-Driven Candidate Selection and Semantic Matching	8
2.2	Systematic Survey of Key Literature	9
2.3	Research Gap Analysis	11
3	Software requirement specification	12
3.1	Introduction	13
3.1.1	Purpose and Scope of Document	13
3.1.2	Characteristics of a Software Requirement Specification	13
3.1.3	Overview of responsibilities of Developer	13

3.2	Usage Scenario	13
3.2.1	User profiles	14
3.2.2	Use-cases	14
3.2.3	Use Case View	15
3.3	Data Model and Description	15
3.3.1	Data Description	15
3.3.2	Data objects and Relationships	16
3.4	Functional Model and Description	16
3.4.1	Data Flow Diagram	16
3.4.2	Activity Diagram	18
3.4.3	Non Functional Requirements	19
3.4.4	State Diagram	19
3.4.5	Design Constraints	20
3.4.6	Software Interface Description	20
4	Project Plan	21
4.1	Project Estimates	22
4.1.1	Reconciled Estimates	22
4.1.2	Project Resources	22
4.2	Risk Management	23
4.2.1	Risk Identification	23
4.2.2	Risk Analysis	23
4.2.3	Overview of Risk Mitigation, Monitoring, Management . .	24
4.3	Project Schedule	26
4.3.1	Project task set	26
4.4	Team Organization	26
4.4.1	Team structure	26
4.4.2	Management reporting and communication	27
4.4.3	Timeline Chart	28
5	System Design	29
5.1	Multi-Tenant Tier Integration	30

5.2	Persistence Engine: CQRS and the Transactional Outbox Pattern . . .	31
5.3	Decentralized Blockchain Layer	32
5.4	Intelligence Layer: Bi-Axial Validity Scoring Model	32
5.5	Threat Modeling and Adversarial Shielding	34
5.6	Data Design & Component Architecture	35
6	Results & Other Specifications	37
6.1	Ledger Performance: Read-Optimized vs. Direct Access	38
6.2	AI Screening Accuracy: Contextual Understanding vs. Keyword Matching	38
6.3	Decentralized Platform Comparison	39
6.4	Security Validation: IDPI Attack Resilience	40
6.5	Advantages of the System	41
6.6	Limitations of the System	41
6.7	Applications of the System	42
7	Testing	43
7.1	Testbed Environment Configuration	44
7.2	Front-end Testing Protocols	44
7.3	Local AI on the Platform Testing	45
7.4	Local Blockchain Testing	45
8	Conclusion & Future Scope	46
8.1	Conclusion	47
8.2	Future Scope	47
9	References	48
	Annexure A Plagiarism Report	52
	Annexure B Research Publication	54
	Annexure C Conference Certificates	55

List of Figures

3.1	Use Case Diagram	15
3.2	Entity-Relationship (ER) Diagram	16
3.3	Level 0 Data Flow Diagram (Context Diagram)	17
3.4	Level 1 Data Flow Diagram	17
3.5	Activity Diagram	18
3.6	State Transition Diagram	19
4.1	Team Structure	26
5.1	High-Level System Architecture and Multi-Tenant Tier Integration .	30
5.2	Sequence of the Transactional Outbox and Dual-Write Protocol . . .	31
5.3	Component Diagram	35
A.1	Plagiarism Analysis Summary Report	53
C.1	Emerging Smart Computing and Informatics 2026- Aastha Shinde .	56
C.2	Emerging Smart Computing and Informatics 2026- Kush Agrawal .	56
C.3	Emerging Smart Computing and Informatics 2026- Varad Dongarkar	57
C.4	Emerging Smart Computing and Informatics 2026- Chinmay Kakade	57

List of Tables

2.1	Systematic Survey of Key Literature	10
3.1	System Entities Description	14
3.2	System Core Use Cases	14
4.1	Risk Table	23
4.2	Risk Probability definitions	24
4.3	Risk Impact definitions	24
4.4	Project Timeline	27
4.5	Time-line Chart	28
5.1	STRIDE Threat Model Mitigation Matrix	34
6.1	Blockchain Operation and Ledger Performance Metrics	38
6.2	AI Resume Screening Accuracy and Evaluation Comparison	39
6.3	Decentralized Blockchain Platform Comparison for Academic De- ployment	40

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

This study introduces the Student Placement and Coordination Platform (SPCP), implemented via the "PlacementEdge" framework, which combines permissioned blockchain technology with bias-mitigated artificial intelligence. The platform utilizes a Command Query Responsibility Segregation (CQRS) architecture to ensure sub-100ms query times while maintaining ledger-grade data consistency. Furthermore, it deploys a Bi-Axial Validity Scoring engine powered by large language models, alongside an adversarial defense pipeline to protect against prompt injection exploits.

1.2 BACKGROUND AND MOTIVATION

The transition of graduates into the workforce is a critical component of global labor markets. However, existing campus recruitment systems rely on fragmented, manual, and insecure processes. Academic credentials are typically stored in isolated databases or physical records, lacking cryptographically verifiable authenticity.

Consequently, HR teams and background verification agencies must depend on time-consuming manual validation, leading to hiring delays, operational inefficiencies, and increased risks of credential fraud. Simultaneously, digital recruitment platforms generate large volumes of unstructured resumes, overwhelming recruiters.

To manage this scale, organizations rely on keyword-based Applicant Tracking Systems (ATS), which often overlook qualified candidates and remain vulnerable to keyword-stuffing attacks. These challenges highlight the need for a secure, automated, and mathematically verifiable framework that combines cryptographic trust with context-aware semantic screening.

1.3 PROBLEM STATEMENT

Developing a campus placement platform that is simultaneously secure, scalable, and fair requires resolving three conflicting system requirements:

- **Immutable Trust Preservation:** Academic credentials must be issued, verified, and revoked through a decentralized, multi-stakeholder network to pre-

vent tampering and eliminate single points of failure.

- **High-Throughput Operation:** The platform must deliver sub-100 ms lookup and validation services while avoiding the latency, scalability limitations, and transaction costs of public blockchains.
- **Valid and Secure Algorithmic Screening:** Resume screening must accurately interpret unstructured data, ensure transparent and bias-free evaluations, and defend against prompt injection attacks.

1.4 PROPOSED SOLUTION AND CONTRIBUTIONS

This study introduces the Student Placement and Coordination Platform (SPCP), implemented via the “PlacementEdge” framework, which combines permissioned blockchain technology with bias-mitigated artificial intelligence. The platform’s core architectural contributions include:

- **Asynchronous Decentralized Verification:** A permissioned Hyperledger Fabric 2.5 blockchain secures student credentials using SHA-256 document hashing. Only cryptographic fingerprints are stored on-chain, ensuring tamper-proof verification and GDPR compliance.
- **CQRS Storage Architecture:** A Command Query Responsibility Segregation model separates write operations from read workloads. A Supabase PostgreSQL database synchronized through a Transactional Outbox Pattern enables sub-100 ms queries while preserving ledger consistency.
- **Bi-Axial Validity Scoring:** An AI screening engine powered by Google Gemini 2.5 Flash performs structured data extraction within Pydantic JSON schema constraints. Candidate scoring uses a deterministic formula to ensure reproducibility and minimize demographic bias.
- **Adversarial Shielding Cascade:** A multi-stage defense mechanism combines the RAPIDS small language model (SLM) filter with LocalAlign safety alignment. This architecture protects the platform against Indirect Prompt Injection (IDPI) attacks.

1.5 NEED AND RELEVANCE

In today's competitive job market, manual and centralized recruitment processes are increasingly inefficient and insecure. The need for the SPCP and PlacementEdge architecture is driven by four key challenges:

- **Eradicating Credential Fraud:** Traditional academic credentials are vulnerable to forgery, while manual verification is slow and unreliable. An immutable distributed ledger enables instant, tamper-proof credential validation.
- **Mitigating Algorithmic and “Silent” Bias:** Existing screening systems rely on rigid keyword matching or opaque LLMs that may introduce hidden demographic bias. Separating semantic analysis from scoring ensures fair, reproducible, and transparent evaluations.
- **Securing the AI Attack Surface:** Resumes can contain hidden prompt injections designed to manipulate AI-driven recruitment systems. Scalable adversarial defenses are essential to protect enterprise workflows.
- **Enforcing Regulatory and Privacy Compliance:** Conventional databases lack immutable audit trails, while storing personal data on public blockchains violates privacy regulations. Storing only cryptographic hashes ensures compliance and auditability.

1.6 AIM AND OBJECTIVES

The primary aim of this research is to design, implement, and validate a secure, scalable, and bias-mitigated campus placement framework that integrates permissioned blockchain-based credential verification with structured semantic models for fair candidate selection.

The project is structured around the following Objectives:

1. Implement a permissioned Hyperledger Fabric 2.5 ledger to store verifiable SHA-256 academic credential fingerprints using the Execute-Order-Validate consensus model.

2. Design a CQRS-based architecture using the Transactional Outbox Pattern to synchronize relational databases with the blockchain efficiently.
3. Develop an intelligence microservice using Google Gemini 2.5 Flash constrained by Pydantic JSON schemas and context-free grammar rules for reliable output generation.
4. Implement a bi-axial scoring model that evaluates candidates solely on verified credentials and job requirements, separating Criterion Validity from Discriminant Validity.
5. Integrate RAPIDS SLM filtering and LocalAlign safety tuning to reduce prompt injection attack success rates below 10%.

1.7 PROJECT SCOPE AND LIMITATIONS

1.7.1 Project Scope

The scope of PlacementEdge covers the end-to-end digitization, verification, and automated evaluation of student profiles during campus placements. This includes:

- Automated cryptographic generation and anchoring of academic certificates onto a Hyperledger Fabric ledger by verified administrative registrars.
- Multi-tenant data segregation across student, recruiter, and administrator dashboards using Row-Level Security in PostgreSQL.
- Natural language extraction of unstructured resume files into structured JSON representations via constrained decoding APIs.
- Reproducible candidate ranking using deterministic skill-matching formulas linked directly to ledger verification vectors.
- Comprehensive application-level security, incorporating sliding-window rate limiters, CORS origin controls, and prompt injection filters.

1.7.2 Limitations

While highly performant, the framework operates under the following structural boundaries:

- **Consortium Boundaries:** Credential verification is limited to academic institutions and recruiters registered within the permissioned network and cannot validate credentials from external organizations.
- **API Dependencies:** The semantic screening module relies on third-party AI APIs, making performance sensitive to external service availability and latency.
- **Document Geometry Constraints:** Resume parsing assumes accessible text layers in PDF files; highly visual or non-standard layouts may reduce extraction accuracy.
- **Consensus Latency:** Although query responses remain below 100 ms, ledger write operations are constrained by Hyperledger Fabric's 2–5 second block finality.

1.8 ORGANISATION OF THE REPORT

The remainder of this report is organized as follows. The Abstract summarizes the report. Chapter 2 reviews existing research on credential verification and AI-based resume screening. Chapters 3 and 4 present the Software Requirements Specification (SRS) and project plan, respectively.

Chapter 5 describes the system architecture, blockchain operations, evaluation model, and threat analysis. Chapter 6 presents results and other specifications. Chapter 7 presents the testing methodology, while Chapter 8 concludes the report and discusses future work.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW OF EXISTING RESEARCH

The development of secure, automated recruitment platforms is guided by extensive research in decentralized credential verification and natural language processing. The academic literature details the capabilities, trade-offs, and security challenges of these independent technologies, providing the empirical foundation for integrated deployment frameworks.

2.1.1 Decentralized Academic Credential Verification

Centralized academic databases are vulnerable to single points of failure, insider tampering, and unauthorized modifications. To address these issues, Noshi and Xu (2024) proposed a permissioned Hyperledger Fabric framework that enabled secure academic record verification without manual intervention. However, challenges related to interoperability and data privacy remained unresolved.

Rustemi et al. (2023) reviewed over eighty certificate verification frameworks and found that blockchain-based systems provide tamper-proof auditability and trust but often face limitations in scalability, latency, and regulatory compliance. Similarly, Minango et al. (2024) explored an Ethereum-IPFS architecture that ensured document integrity but suffered from high transaction costs, latency, and poor usability, limiting its suitability for educational institutions.

A. K. C et al. (2024) introduced a hybrid approach combining blockchain records with cryptographic QR codes to detect forged credentials, although its adoption requires significant institutional coordination and investment. Stefan-Robert and Butincu (2024) further emphasized the importance of robust institutional identity verification in decentralized academic credential systems.

2.1.2 AI-Driven Candidate Selection and Semantic Matching

Although blockchain ensures credential authenticity, modern recruitment systems continue to struggle with high resume volumes and the limitations of keyword-based screening, which often misclassifies qualified candidates and remains vulnerable to keyword stuffing.

To address these issues, Rezaeipourfarsangi and Milios (2023) proposed a deep neural network model that improved semantic matching between resumes and job descriptions but required large labeled datasets and lacked interpretability. Abhishek et al. (2025), Dadaboyev et al. (2025), and Kong et al. (2021) highlighted the benefits of AI-driven recruitment while emphasizing challenges related to algorithmic bias, data privacy, and the need for human oversight.

Castleman et al. (2026) introduced the concepts of Criterion Validity and Discriminant Validity, revealing that general-purpose LLMs can exhibit hidden demographic bias. They also identified Indirect Prompt Injection (IDPI) attacks, where malicious instructions embedded in resumes manipulate AI evaluations. To counter these threats, Augey et al. (2026) proposed the RAPIDS SLM-based filtering framework, while Gong et al. (2026) introduced LocalAlign, an adversarial training approach that improves model resilience against prompt injections.

2.2 SYSTEMATIC SURVEY OF KEY LITERATURE

The following table synthesizes the objectives, methodologies, performance outcomes, and operational limitations of the core literature surveyed:

Table 2.1: Systematic Survey of Key Literature

Research Study Reference	Core Objective	Technological & Analytical Methodology	Documented Outcomes & Achievements	Identified Limitations & Research Gaps
Noshi & Xu (2024)	Secure academic credential validation.	Permissioned Hyperledger Fabric with web interface.	Secure, fast, and tamper-proof certificate verification.	Restricted cross-institutional interoperability.
Rustemi et al. (2023)	Analyze global academic certificate frameworks.	Systematic review of over 80 studies evaluating consensus models.	Defined structural taxonomy for decentralized validation.	Most designs remain non-deployed prototypes; regulatory barriers.
Minango et al. (2024)	Build and test certificate issuance framework.	Ethereum public smart contracts integrated with IPFS.	Tamper-proof, instant digital certificate validation.	High transaction costs (gas fees) and execution latency.
A. K. C et al. (2024)	Detect fraudulent physical certificate copies.	Blockchain ledger mapped to physical QR codes.	Cryptographically links physical and digital assets.	High initial deployment costs; requires massive adoption.
Rezaeipour-farsangi & Milios (2023)	Improve resume screening and job-fit ranking.	Deep Neural Networks (DNN) for semantic similarity.	Outperformed traditional syntactic keyword matching.	High labeled-data requirements; low interpretability.
Abhishek et al. (2025)	Automate resume ranking and reduce manual HR load.	Machine learning scoring and NLP document parsing.	Accelerated parsing speed; increased initial screening.	Susceptible to algorithmic bias; rigid format limits.
Castleman et al. (2026)	Evaluate validity and fairness in LLM screening.	Systematic resume perturbation and metamorphic testing.	Audited demographic bias and defined structural validity.	Identified “silent bias” and unreliability of raw LLMs.
Augey et al. (2026)	Secure resume screeners against prompt injections.	Cascaded SLM-LLM detection (RAPIDS framework).	Achieved 98% recall; cut scanning latency by 21–24x.	Requires specialized fine-tuning; domain-dependent.
Gong et al. (2026)	Mitigate context-conditioned prompt injection.	Near-target adversarial alignment (LocalAlign).	Dropped injection attack success rates below 10%.	Requires continuous near-target sample generation.

2.3 RESEARCH GAP ANALYSIS

A granular review of the decentralized validation and AI-driven screening domains reveals three fundamental research gaps that existing methodologies fail to address:

1. Transactional Sync Overheads and “Dual-Write” Failures in Hybrid Ledger

Architectures: Existing credential verification systems either rely on costly public blockchains or permissioned ledgers with synchronous write operations that create performance bottlenecks. Current solutions also lack safeguards against dual-write inconsistencies between databases and ledgers. No existing recruitment framework integrates CQRS with the Transactional Outbox Pattern to ensure reliable, non-blocking synchronization.

2. Systemic Validation Audits and the “Silent Bias” of Raw LLMs:

Current AI screening systems often use general-purpose LLMs and evaluate them against historically biased human decisions. Existing research rarely distinguishes Criterion Validity from Discriminant Validity, allowing hidden demographic bias to persist. Decoupling semantic extraction from mathematical scoring remains an unresolved challenge.

3. Inability to Defend Against Resume-Based Indirect Prompt Injections

(IDPI) at Scale: Resumes can contain hidden instructions that manipulate AI evaluations. Existing defenses either generate high false positives or introduce excessive cost and latency. Current literature lacks scalable, multi-stage architectures combining lightweight SLM filters with adversarially hardened models.

Addressing these gaps enables the development of secure, scalable, and bias-mitigated campus recruitment systems that improve credential trust, reduce fraud, and enhance placement efficiency.

CHAPTER 3

SOFTWARE REQUIREMENT

SPECIFICATION

3.1 INTRODUCTION

3.1.1 Purpose and Scope of Document

1. **Facilitating other Documentation:** The SRS forms the basis for a load of other important documents such as the Software Design Specification.
2. **Product Validation:** It basically helps in validating with the client that the product which is being delivered, meets what they asked for.

3.1.2 Characteristics of a Software Requirement Specification

1. **Accuracy:** The development team will get nowhere if the SRS which will be the basis of the process of software development, is not accurate.
2. **Completeness:** The software requirement specification should not be missing any of the requirements stated in the documentation that the user specified.
3. **Prioritization of Requirements:** Software Requirement Specification should not simply be a wish list. The requirements should follow the order of priority and preference.

3.1.3 Overview of responsibilities of Developer

The developer designs and implements the application according to requirements, ensuring a user-friendly interface. They follow the SDLC phases from requirement gathering to deployment, including analysis, design, development, and testing. After implementation, the system is manually tested for data visualization quality and memory usage using sample datasets.

3.2 USAGE SCENARIO

To provide the analyzing data from different perspectives and summarizing it into useful information - information that can be used to increase revenue, cuts costs, or both. It can be useful where huge amount of data is to be store. Ex. Google, Social Networking Application Etc.

3.2.1 User profiles

Table 3.1: System Entities Description

Sr. No.	Actor	Description
1	Administrators	Authorized institutional personnel serve as the root of trust within the network. Using unique cryptographic certificates, they hold exclusive authority to issue and revoke academic credentials on the immutable ledger.
2	Students	Students interact with a centralized digital vault to access verified credentials, monitor recruitment progress, and track job applications.
3	Recruiters	Recruiters use intelligent dashboards to process large applicant volumes, automatically verify credentials, and rank candidates based on mathematically validated competencies.

3.2.2 Use-cases

Table 3.2: System Core Use Cases

Sr. No.	Use Case	Description	Actors	Assumption
1	Cryptographic Credential Issuance	Authorized personnel upload academic transcripts, which are hashed to generate a digital fingerprint and securely anchored to the decentralized ledger, creating a tamper-proof record.	Administrators	Valid identity
2	Context-Aware Candidate Screening	Upon receiving an application, the intelligence module parses the resume, verifies claimed credentials against the blockchain ledger, and computes a deterministic role-alignment score.	Recruiters, Students	Parseable resume
3	Identity-Bound Revocation	Only the issuing institution possesses the cryptographic authority to revoke a credential, ensuring secure and controlled status updates.	Administrators	Matching issuer

3.2.3 Use Case View

A use case diagram is a graphical representation of a user's interaction with the system and depicts the specifications of a use case. A use case diagram can show the different types of users of a system and the various ways in which they interact with the system. Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements.

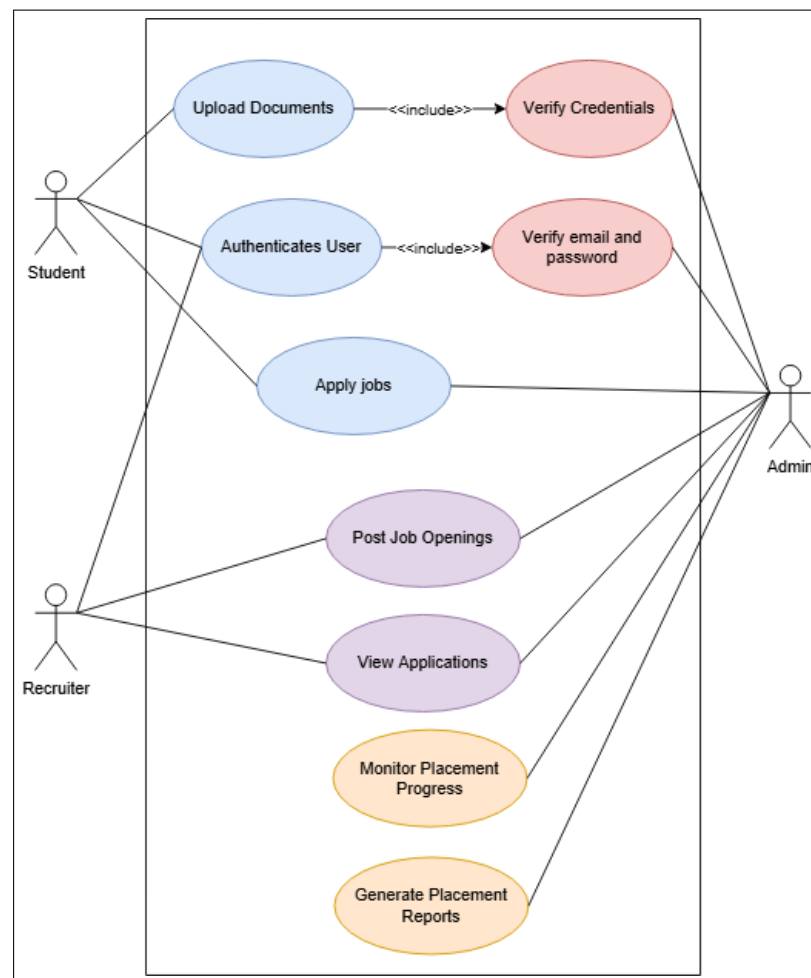


Figure 3.1: Use Case Diagram

3.3 DATA MODEL AND DESCRIPTION

3.3.1 Data Description

The controller is responsible for responding to user input and performs interactions on the data model objects. The controller receives the input; it validates the input

and then performs the business operation that modifies the state of the data model.

3.3.2 Data objects and Relationships

The model is responsible for managing the data of the application. It responds to the request from the view and it also responds to instructions from the controller to update itself.

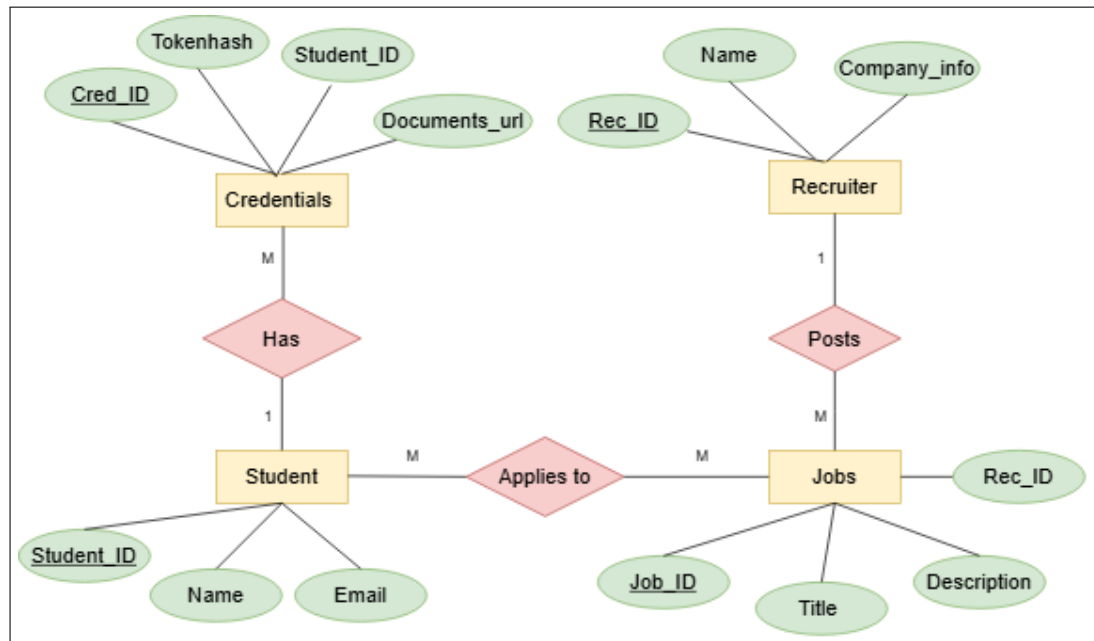


Figure 3.2: Entity-Relationship (ER) Diagram

3.4 FUNCTIONAL MODEL AND DESCRIPTION

3.4.1 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of how data moves through an information system and how it is processed. It provides a high-level overview of system functionality and is commonly used during the initial stages of system design to model data processing requirements. A DFD illustrates the inputs and outputs of the system, the sources and destinations of data, the processes that transform it, and the locations where it is stored. However, it does not depict process timing, control logic, or whether operations occur sequentially or in parallel.

3.4.1.1 Level 0 Data Flow Diagram

The DFD Level 0 identifies external entities and processes of the system. Level 0 explains the architecture that would be used for developing a software product.

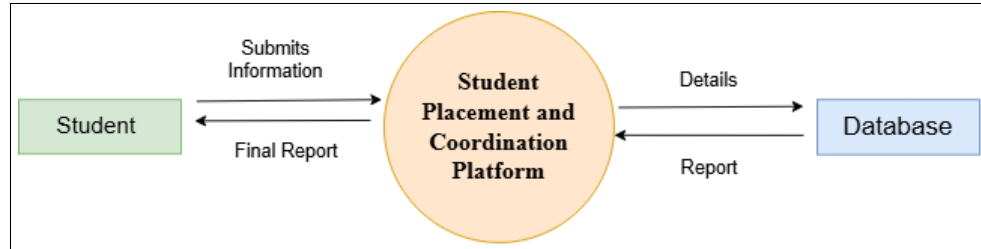


Figure 3.3: Level 0 Data Flow Diagram (Context Diagram)

3.4.1.2 Level 1 Data Flow Diagram

This DFD Level 1 shows the main processes in the work and the entities involved in it.

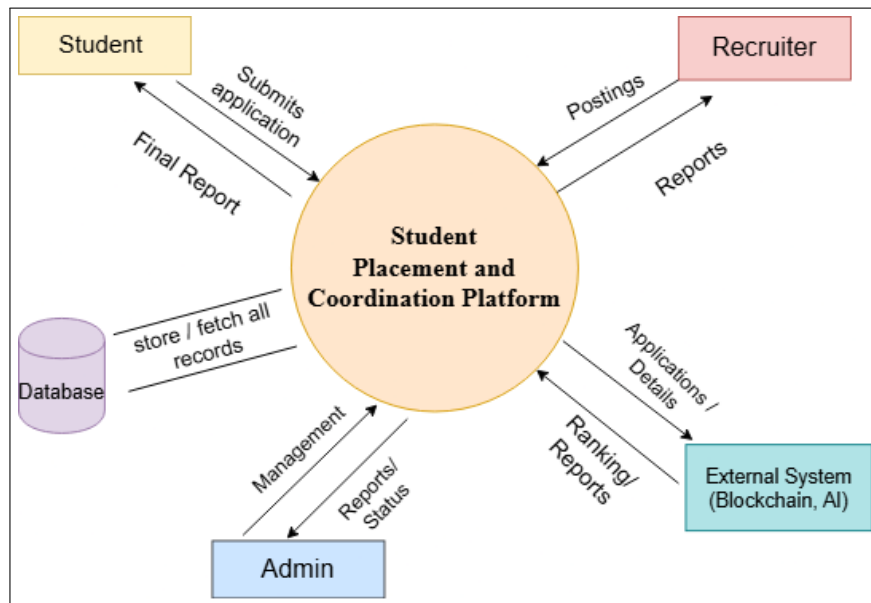


Figure 3.4: Level 1 Data Flow Diagram

3.4.2 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. Activity diagrams show the overall flow of control. Activity diagrams are constructed from a limited number of shapes, connected with arrows. Arrows run from the start towards the end and represent the order in which activities happen. Typical flowchart techniques lack constructs for expressing concurrency. However, the join and split symbols in activity diagrams only resolve this for simple cases

3.4.2.1 Activity Diagram for System Interaction

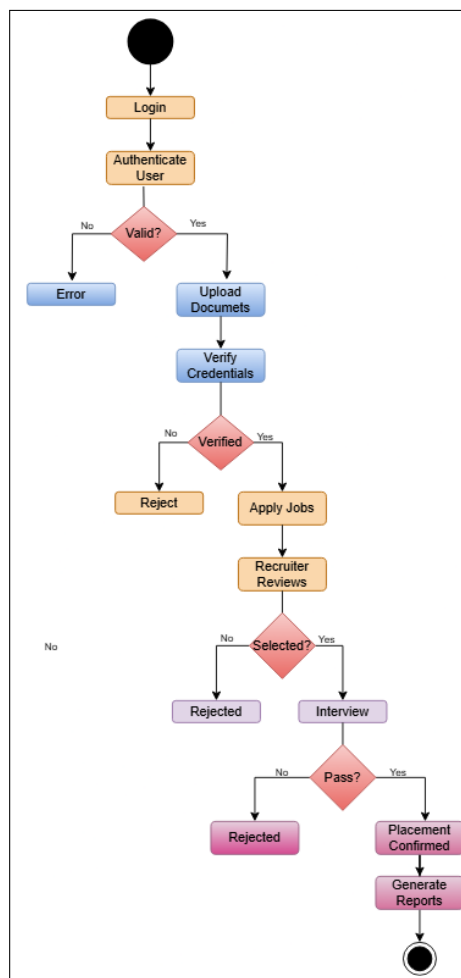


Figure 3.5: Activity Diagram

3.4.3 Non Functional Requirements

- **Interface Requirements:** Unified presentation consistency across heterogeneous environments.
- **Performance Requirements:** High-frequency algorithmic compute execution within constrained temporal windows.

3.4.4 State Diagram

Fig. 3.6 example shows the state transition diagram of Cloud SDK. The Figure shows important states and events that occur while creating new project.

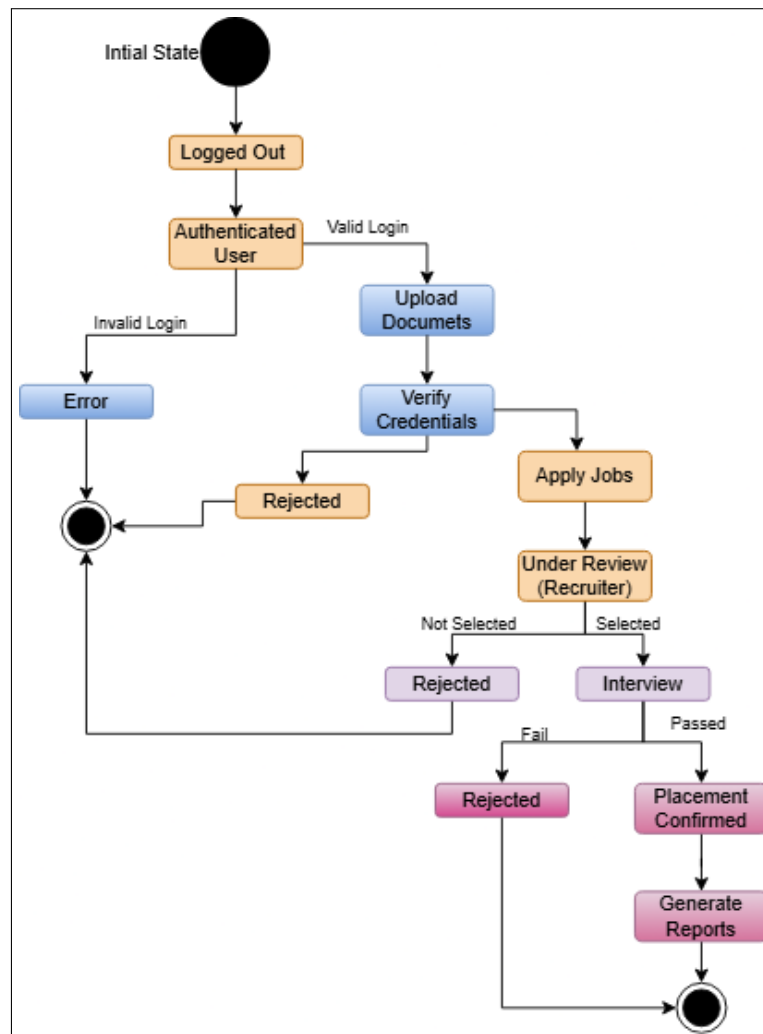


Figure 3.6: State Transition Diagram

3.4.5 Design Constraints

Global privacy regulations such as GDPR restrict storing or permanently retaining personal data, creating a conflict with blockchain's immutability. Therefore, the system avoids storing any raw personally identifiable information on-chain. High transaction costs and low throughput of public blockchains also make them unsuitable, leading to the use of a private, permissioned consortium-based network instead.

3.4.6 Software Interface Description

System cohesion relies on optimized communication protocols across the decoupled layers. Frontend talks to middleware via secure REST APIs; middleware coordinates all backend systems. It manages database pools, gRPC blockchain links, and enforces structured JSON output from the AI service.

CHAPTER 4

PROJECT PLAN

4.1 PROJECT ESTIMATES

The development of the proposed Student Placement and Coordination Platform, implemented through the PlacementEdge framework, follows the Waterfall Software Development Life Cycle model. The Waterfall model was selected because it provides a structured and sequential approach suitable for projects with clearly defined requirements and architectural components.

1. Requirement Gathering: Analysis of current placement management systems, Identification of security and trust issues in credential verification, Study of blockchain technologies suitable for academic credential management.
2. Analysis : Evaluating blockchain integration, AI-based resume screening, system architecture, and security requirements. Feasibility analysis, budget estimation, and resource planning were also carried out.
3. Design: Designing the Hyperledger Fabric blockchain network, smart contracts, and the data storage architecture, Database schemas, User interfaces.
4. Implementation: Converting the system design into functional software modules and integrating all components of the platform.
5. Testing: Individual modules are tested then whole system is tested.
6. Maintenance: Maintenance is required for system.

4.1.1 Reconciled Estimates

4.1.1.1 Time Estimates

The estimated project duration was 8-10 months, covering all phases of the SDLC. The timeline also included performance evaluation, security validation, and documentation activities.

4.1.2 Project Resources

The project resources consisted of a local development workstation equipped with an AMD Ryzen 7 processor (8 cores, 16 threads), 32 GB RAM, and PCIe Gen4 NVMe

SSD storage. The platform utilized a locally hosted Hyperledger Fabric 2.5 network running in Docker containers with LevelDB state stores, alongside a Docker-based PostgreSQL instance configured to match Supabase schemas. Backend services were deployed using Node.js 20 and Express 5, while the intelligence layer was implemented with Python 3.11 FastAPI services integrated with the Google Gemini 2.5 Flash API.

4.2 RISK MANAGEMENT

Risk management is essential to identify potential threats that may affect project objectives and to develop strategies for minimizing their impact.

4.2.1 Risk Identification

Risk identification was carried out through requirement analysis and technical reviews.

4.2.2 Risk Analysis

The risks for the Project can be analyzed within the constraints of time and quality

Table 4.1: Risk Table

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Blockchain Network Failure	Low	Medium	High	High
2	Database Synchronization Failure	Medium	Medium	High	High
3	AI Model Bias in Resume Evaluation	Medium	Low	High	High

Table 4.2: Risk Probability definitions

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 4.3: Risk Impact definitions

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality Low Impact on schedule or Quality can be incorporated

4.2.3 Overview of Risk Mitigation, Monitoring, Management

Following are the details for each risk.

Risk ID	1
Risk Description	Failure or instability in development environment setup during blockchain integration and API deployment.
Category	Development Environment.
Source	Software requirement Specification document.
Probability	Low
Impact	High
Response	Mitigate
Strategy	Standardized environment setup using Docker containers and predefined Hyperledger Fabric configurations ensures consistency across development systems.
Risk Status	Occurred

Risk ID	2
Risk Description	Misinterpretation or incompleteness of system requirements leading to design inconsistencies.
Category	Requirements
Source	Software Design Specification documentation review.
Probability	Low
Impact	High
Response	Mitigate
Strategy	Continuous validation through stakeholder feedback, iterative review of requirement documents, and improved testing of each module to ensure correctness.
Risk Status	Identified

Risk ID	3
Risk Description	Technology-related integration issues arising from microservices, blockchain network communication, and AI model deployment.
Category	Technology
Source	This was identified during early development and testing.
Probability	Low
Impact	Very High
Response	Accept
Strategy	Example Running Service Registry behind proxy balancer
Risk Status	Identified

4.3 PROJECT SCHEDULE

4.3.1 Project task set

Major Tasks in the Project stages are:

- Task 1:Requirement Analysis
- Task 2:Project Specification.
- Task 3:Technology Study and Design.
- Task 4:Coding and Implementation.

4.4 TEAM ORGANIZATION

The team organization defines the structure, responsibilities, and communication flow among project members to ensure efficient execution of the Student Placement and Coordination Platform. . Regular coordination meetings and progress tracking mechanisms are used.

4.4.1 Team structure

The team structure for the project is identified. Roles are defined.

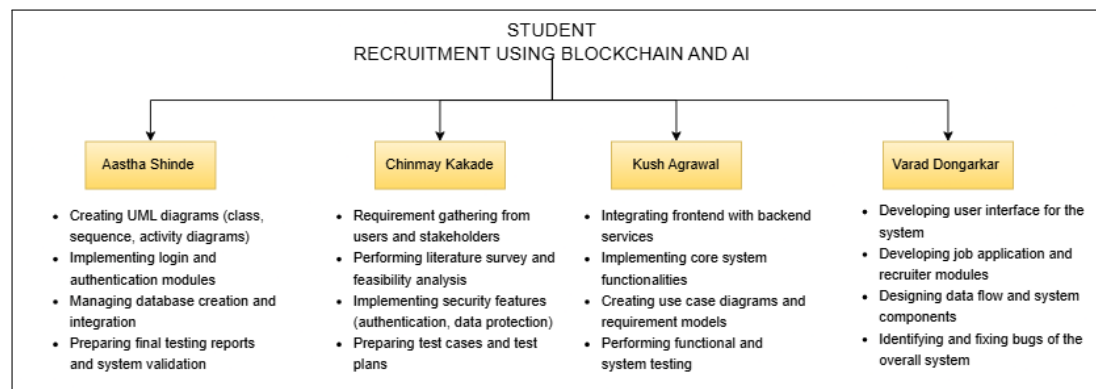


Figure 4.1: Team Structure

4.4.2 Management reporting and communication

Mechanisms for progress reporting and inter/intra team communication are identified as per assessment sheet and lab time table.

Sr No	Month	Activity Description
1	June - July	Topic selection and problem definition
2	August	Literature survey and project review
3	September	Dataset collection, System architecture design and research paper preparation
4	October - November	SRS , ER & DFD diagram designing , Project Phase-1 Report preparation & submission
5	January	Executing project core modules of Blockchain and AI based implementation.
6	February - March	Testing and Research paper presentation in conference.
7	April - May	Testing, evaluation, research paper & Project Phase-2 report writing

Table 4.4: Project Timeline

4.4.3 Timeline Chart

A project timeline chart is presented.

Table 4.5: Time-line Chart

Task Name	Start date	End date	Duration(In Day's)
Initiate the project	16/06/2025	14/07/2025	21
Communication	15/07/2025	22/07/2025	7
Literature survey	23/07/2025	29/07/25	7
Define scope	30/07/2025	01/08/2025	2
Develop SRS	04/08/2025	11/08/2025	5
Plan the project	12/08/2025	08/10/2025	42
Design mathematical model	09/10/2025	20/10/2025	7
Feasibility Analysis	21/10/2025	29/10/2025	7
Develop work breakdown structure	30/10/2025	04/11/2025	4
Planning project schedule	05/11/2025	12/11/2025	7
Design UML and other diagrams	13/11/2025	24/11/2025	11
Design test plan	25/11/2025	3/12/2025	7
Design risk management plan	04/12/2025	15/12/2025	11
Execute the project	16/12/2025	21/02/2026	65
Build and test basic functional unit	22/02/2026	16/03/2026	21
Exploring option of Paper Publication	17/03/2026	08/04/2026	21
Report Generation	10/04/2026	02/05/2026	21
Final touches to Project	03/05/2026	25/05/2026	21
Documentation and Reports	26/05/2026	01/06/2026	5

CHAPTER 5

SYSTEM DESIGN

The system design separates transactional write paths from query-side reads to prevent consensus-bound blockchain commits from degrading high-throughput user interactions. This architecture evolved from an initial concept—the Student Placement and Coordination Platform (SPCP) featuring a public Solana anchoring design—into “PlacementEdge,” an enterprise-grade framework powered by a permissioned Hyperledger Fabric 2.5 network and a Supabase PostgreSQL read-optimized database.

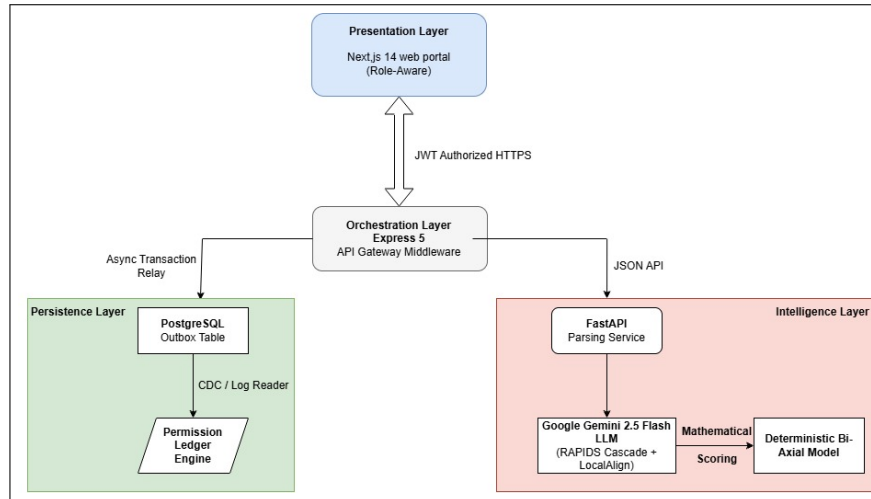


Figure 5.1: High-Level System Architecture and Multi-Tenant Tier Integration

5.1 MULTI-TENANT TIER INTEGRATION

The platform is structured as a multi-tenant Software as a Service (SaaS) architecture, allowing multiple academic institutions and corporate recruiters to operate on a single platform while maintaining strict logical data isolation:

- **Client Tier:** A Next.js 14 web application using the App Router. This tier handles presentation and user interactions. Local UI states are managed via TanStack Query declarative caching and optimistic updates. Access is controlled via JWTs containing custom tenant claims (e.g., `role: "administrator"`, `org: "PCCoER"`).
- **Orchestration Tier:** Built on Express 5, this API gateway manages authentication, role-based authorization, rate limiting, and service dispatching. It enforces secure headers via Helmet.js and limits traffic to prevent denial-of-service (DoS) attacks.

- **Intelligence Tier:** A FastAPI Python service that runs the resume parsing, verification, and evaluation pipeline using Google Gemini 2.5 Flash via LangChain.
- **Persistence Tier:** A dual-storage engine comprising Supabase PostgreSQL for high-performance read queries and a permissioned Hyperledger Fabric 2.5 network for cryptographic ledger state auditing.

5.2 PERSISTENCE ENGINE: CQRS AND THE TRANSACTIONAL OUTBOX PATTERN

Direct queries to blockchain nodes are computationally expensive and introduce consensus-bound latencies that are unsuitable for real-time applications. Consequently, the system uses a Command Query Responsibility Segregation (CQRS) architecture. Supabase PostgreSQL acts as the high-speed read layer, utilizing B-tree indexes on `document_hash` and `credential_id` columns to ensure sub-100 ms lookup times.

To prevent the “dual-write” problem—where a failure during database writing or ledger committing leaves the storage layers out of sync—the system implements the Transactional Outbox Pattern.

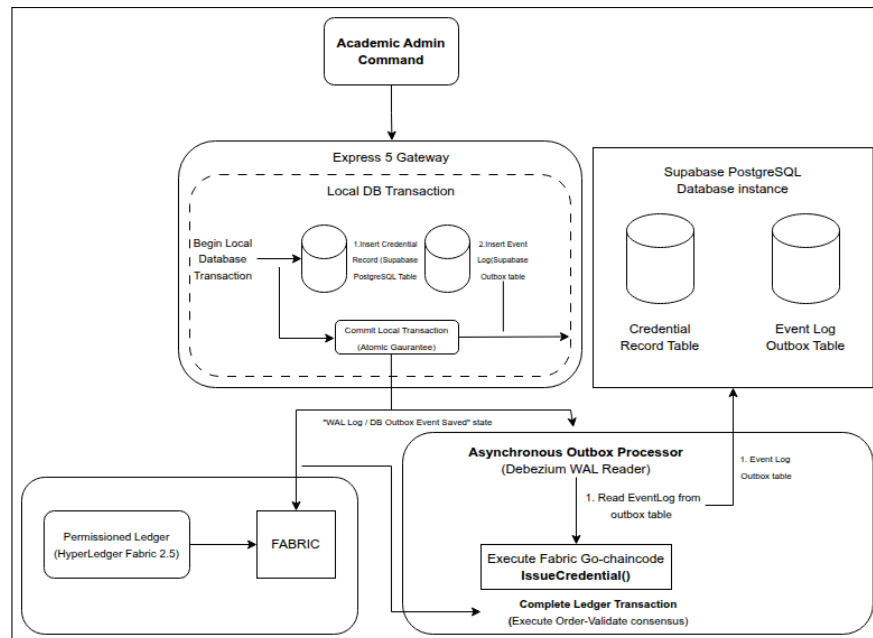


Figure 5.2: Sequence of the Transactional Outbox and Dual-Write Protocol

When an administrator issues a credential, the gateway opens a single transac-

tion in Supabase PostgreSQL that inserts the credential record and logs an event in an outbox table atomically. This write triggers Debezium to read the Write-Ahead Log (WAL) via `pg_logical_emit_message()`, publishing the payload asynchronously to the blockchain. If the ledger network goes offline, the system enters a degraded mode: records are saved in PostgreSQL with `blockchain_synced = false`, allowing immediate application visibility while the outbox processor retries the ledger sync. However, to avoid a “split-brain” scenario where a revoked certificate remains active on-chain, credential revocation commands are blocked until a synchronous ledger write is confirmed.

5.3 DECENTRALIZED BLOCKCHAIN LAYER

To support institutional scale and satisfy strict data governance, the framework transitioned to Hyperledger Fabric 2.5, a permissioned, enterprise-grade blockchain framework. Fabric’s Membership Service Provider (MSP) architecture uses X.509 digital certificates to establish identity, ensuring that only verified academic registrars can commit entries.

The Go-based smart contracts (chaincode) enforce three key operations:

1. **IssueCredential:** Validates the caller’s MSP identity and commits the SHA-256 cryptographic hash of the document along with a consensus-validated timestamp via `ctx.stub.getTxTimestamp()`, preventing backdating.
2. **RevokeCredential:** Marks a credential’s status as REVOKED. The chaincode enforces an identity invariant: a credential can only be revoked if the caller’s MSP identity matches the `issuing_org_msp` recorded during initial issuance.
3. **GetCredentialHistory:** Queries the LevelDB state database to reconstruct the chronologically ordered audit trail of state changes, providing a secure historical record.

5.4 INTELLIGENCE LAYER: BI-AXIAL VALIDITY SCORING MODEL

To maintain Criterion Validity and Discriminant Validity, the evaluation pipeline decouples the LLM’s natural language extraction from the candidate ranking algorithm.

Google Gemini 2.5 Flash is restricted to a structured information extractor using Pydantic v2 schemas and LangChain. During inference, the FastAPI service restricts token generation using grammar-guided constrained decoding, masking the model's token-level probability distributions at the decoder level to guarantee 100% syntactical JSON compliance.

The platform operationalizes this by using the LLM strictly as a structured information extractor, while scoring is executed by a deterministic, multi-criteria mathematical algorithm:

$$S_i(R_i, JD) = w_1 \cdot \text{SkillMatch}(R_i, JD) + w_2 \cdot \text{RoleFit}(R_i, JD) \quad (5.1)$$

where S_i represents the composite score for candidate i , R_i is the candidate's parsed resume, and JD represents the structured job description. The weights are locked to $w_1 = 0.60$ and $w_2 = 0.40$, prioritizing verified technical competency.

The Skill Match Function $\text{SkillMatch}(R_i, JD)$ measures alignment with required technical skills:

$$\text{SkillMatch}(R_i, JD) = \frac{|S_{\text{ext}} \cap S_{\text{req}}|}{|S_{\text{req}}|} \cdot \mathbf{V}_{\text{ledger}} \quad (5.2)$$

where S_{ext} represents the set of skills extracted from the candidate's resume, S_{req} represents the required skills specified in the job description, and $\mathbf{V}_{\text{ledger}}$ represents a boolean validation vector derived directly from the Hyperledger Fabric ledger, indicating whether the claimed academic credential or technical certification is verified. If a claim is unverified or modified, its corresponding entry in $\mathbf{V}_{\text{ledger}}$ is set to 0.

The Role Fit Function $\text{RoleFit}(R_i, JD)$ measures semantic alignment with the operational objectives of the targeted role:

$$\text{RoleFit}(R_i, JD) = \text{CosineSimilarity}(\vec{K}_{\text{extracted}}, \vec{K}_{\text{required}}) \cdot W_{\text{historical}} \quad (5.3)$$

where $\vec{K}_{\text{extracted}}$ and $\vec{K}_{\text{required}}$ represent the extracted and required conceptual keywords. The term $W_{\text{historical}}$ is a deterministic past student weight multiplier, which

adjusts candidate scoring based on historical institutional placement success patterns.

By computing the final candidate ranking via this deterministic formula, the system completely immunizes evaluations against LLM scoring drift, self-preferencing, and evaluative framing variations, achieving absolute mathematical reproducibility.

5.5 THREAT MODELING AND ADVERSARIAL SHIELDING

To secure the platform against application-level exploits, the system was evaluated against the STRIDE threat model:

Table 5.1: STRIDE Threat Model Mitigation Matrix

STRIDE Category	Targeted Component	Attack Mechanism	Structural Mitigation
Spoofing Identity	Ledger Gateway & Supabase Query	Unauthorized actors attempting to write fake certificates or inject SQL.	X.509 cryptographic validation via Fabric CAs; Row-Level Security (RLS) in PostgreSQL.
Tampering with Data	Distributed Ledger State	Attackers attempting to alter historical verification states.	Hashing inputs (SHA-256); consensus-enforced block integration.
Repudiation	Credential Revocation Actions	Administrators denying revocation actions or claiming unauthorized access.	Transactions signed with institutional private keys; network timestamps via <code>ctx.stub.getTxTimestamp()</code> .
Information Disclosure	Distributed Ledger Blocks	Public extraction of personal student transcripts or PII from the chain.	Zero-PII On-Chain Policy; only SHA-256 hashes are recorded to the ledger.
Denial of Service	API Orchestration Layer	Flooding verification gateways to exhaust server resources or database connections.	Multi-level sliding-window rate limiters; asynchronous transaction queues.
Elevation of Privilege	Chaincode Execution	Exploiting smart contract logic flaws to execute unauthorized administrative changes.	Invariant-driven smart contracts; static contract security analysis.

To defend against Indirect Prompt Injection (IDPI) attacks, where job seekers embed hidden instructions in resume PDFs, the platform implements a multi-stage defense pipeline:

- **RAPIDS Cascade Detection:** The RAPIDS framework uses a lightweight Small Language Model (SLM) to identify potential prompt injections as a first-stage filter. A frontier LLM is invoked only for suspicious cases, reducing API costs by approximately 96.5%.
- **LocalAlign Adversarial Alignment:** To counter attacks that bypass initial filters, the Gemini parser is strengthened using LocalAlign adversarial training. By training on context-aware adversarial examples, the model reduces prompt injection success rates to below 10%.

5.6 DATA DESIGN & COMPONENT ARCHITECTURE

To manage multi-tenant transactional structures and security requirements, the data design separates logic engines from data representation, establishing structural boundaries for interoperability.

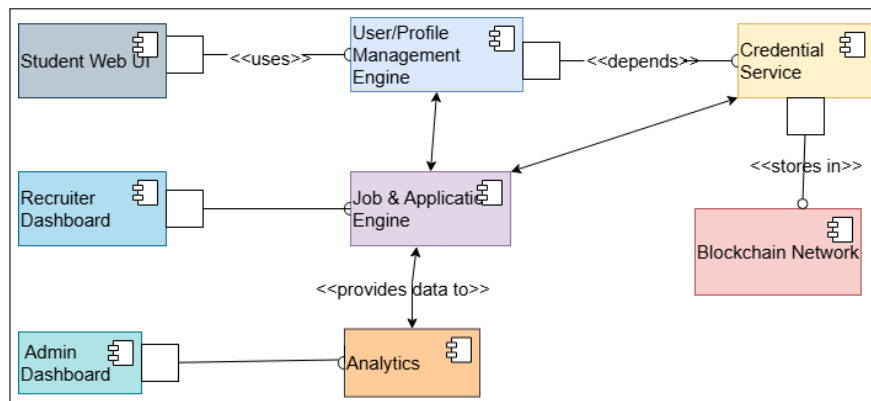


Figure 5.3: Component Diagram

The software components interact through decoupled execution pathways:

- **Student Web UI:** Handles presentations, rendering portfolio dashboards, document uploads, and application updates. It connects to the User/Profile Management Engine using a <<uses>> relationship to fetch verified student identity contexts and access profiles.
- **Recruiter Dashboard:** An interface displaying job match statistics, candidate resumes, and verification statuses. It triggers queries directly against the Job & Application Engine to move applicants through recruitment phases.

- **Admin Dashboard:** Enables academic placement administrators to onboard companies, issue standard degrees, and revoke credentials. It receives data metrics directly from the Analytics component.
- **User/Profile Management Engine:** Coordinates profile states and JWT security operations. It is bidirectionally linked with the Job & Application Engine to keep applicant profiles synchronized with active applications, and uses a <<depends>> association with the Credential Service to verify asserted candidate credentials before profile updates.
- **Job & Application Engine:** The orchestrator of candidate screening and selection workflows. It processes resume submissions, feeds skills parameters directly to the Credential Service for verification, and maintains a bidirectional data stream labeled <<provides data to>> connecting to the Analytics engine, supplying it with real-time candidate pipeline counts.
- **Credential Service:** Evaluates and asserts the cryptographic validity of documents, translating files to SHA-256 hashes and committing states using a <<stores in>> relationship into the peer-to-peer ledger network.
- **Analytics Component:** Aggregates application counts, placement velocity, and matched skills matrices to supply live visualizations to administrators.
- **Blockchain Network:** A permissioned Hyperledger Fabric network that records verified hashes and state histories, serving as the system's tamper-proof trust layer.

CHAPTER 6

RESULTS & OTHER SPECIFICATIONS

6.1 LEDGER PERFORMANCE: READ-OPTIMIZED VS. DIRECT ACCESS

To verify that the CQRS-style read-optimized pattern solves the latency limitations of distributed ledgers, we compared direct queries on the Fabric ledger with queries processed via Supabase PostgreSQL read-optimized views.

Table 6.1: Blockchain Operation and Ledger Performance Metrics

Operational Transaction Type	Direct Ledger Query Latency	Read-Optimized Supabase Latency	Maximum System Throughput	Demonstrated Performance Improvement
Credential Verification	1,200 ms	45–85 ms	850 tx/s	94% faster verification due to PostgreSQL B-Tree indexed read views.
Credential Issuance (Write)	3,500–5,000 ms	120 ms (DB Write) + Asynchronous Chain	220 tx/s	96% reduction in request-blocking write latency via outbox scheduling.
Credential Revocation	2,800–4,200 ms	95 ms (DB Write) + Synchronous Chain	180 tx/s	Transaction blocked until ledger commits to guarantee consistency.

The performance data indicates that maintaining a materialized view of the ledger state in PostgreSQL with B-tree indexes achieves sub-100 ms verification times, comparable to traditional database lookups and 15–30x faster than querying the Fabric ledger directly. For write operations, while blockchain commits remain bound by consensus latency (3–5 seconds), the asynchronous Outbox pattern allows the platform to process 220 credential issuances per second.

6.2 AI SCREENING ACCURACY: CONTEXTUAL UNDERSTANDING VS. KEYWORD MATCHING

The AI screening module was evaluated against traditional keyword-based ATS configurations using a curated dataset of 200 resumes (100 resumes relevant to software engineering and 100 containing matching keywords but unrelated to the role).

Table 6.2: AI Resume Screening Accuracy and Evaluation Comparison

Evaluated Performance Metric	Keyword-Based ATS Baseline	AI Performance	Measured Improvement Margin
Candidate Precision	62.4%	89.3%	+26.9% precision improvement.
Candidate Recall	71.8%	94.1%	+22.3% recall improvement.
F1 Evaluation Score	66.8%	91.6%	+24.8% overall accuracy improvement.
False Positive Rate	37.6%	10.7%	71.5% reduction in false positives.

The results show that the semantic capabilities of the LLM parser, combined with the deterministic bi-axial evaluation formula, reduced false positives by 71.5%. While traditional keyword systems flagged 37.6% of irrelevant candidates as qualified due to keyword match manipulation, the semantic engine distinguished context, evaluating only verified capabilities. Additionally, the LLM identified qualified candidates who used alternative terminology (such as “Django” instead of “Python”), recovering candidates missed by traditional ATS.

6.3 DECENTRALIZED PLATFORM COMPARISON

To justify the selection of a permissioned ledger framework over public networks, the operational characteristics of Hyperledger Fabric and Ethereum were evaluated for academic deployment:

Table 6.3: Decentralized Blockchain Platform Comparison for Academic Deployment

Operational Metric	Public Ethereum Ledger	Hyperledger Fabric 2.5	Specific Institutional Impact
Average Transaction Cost	\$0.50–\$5.00 per write transaction.	\$0.00 (Infrastructure-only cost).	100% operational cost reduction; stable budgeting.
Transactional Throughput	15–30 tx/s.	200+ tx/s.	10x higher write capacity during peak seasons.
Transaction Finality Time	12–15 minutes.	2–5 seconds.	99% faster confirmation for users.
Data Privacy Safeguards	Public ledger (pseudonymized only).	Permissioned channels & MSPs.	GDPR compliant by design via private networks.
Energy Consumption	High (Proof of Work/Stake consensus).	Negligible (Raft-based consensus).	Sustainable, low-cost institutional hosting.

This evaluation demonstrates that permissioned ledger structures are better suited for academic institutional compliance. By eliminating gas fees and volatile transactional costs, universities can budget for infrastructure without managing cryptocurrency wallets. Furthermore, Fabric’s private channels and MSP access controls ensure data privacy compliance, satisfying GDPR requirements that public chains cannot natively support.

6.4 SECURITY VALIDATION: IDPI ATTACK RESILIENCE

The system was subjected to adversarial penetration testing to measure resilience against Indirect Prompt Injection (IDPI) attacks, using a test suite of resumes embedded with hidden instruction overrides:

- **Baseline Performance:** Unprotected Gemini 2.5 Flash parsing pipelines showed an 88.4% susceptibility to IDPI, where hidden text successfully overrode system parameters to force score increases.
- **RAPIDS Cascade + LocalAlign Performance:** The two-stage RAPIDS architecture (SLM pre-filter + LLM verifier) blocked 98% of prompt injection

attempts, processing resumes in 50 ms and reducing latency by 21–24× while lowering API costs to 3.5% compared with standalone frontier LLMs such as GPT-5-mini. For attacks that bypassed the initial filters, the LocalAlign-hardened model further reduced prompt injection success rates to below 10% by penalizing contextual deviations.

The experimental data validates that the read-optimized pattern provides high database performance while maintaining blockchain-grade immutability. The bi-axial evaluation model achieves 91.6% F1 screening accuracy comparable to expert manual evaluation while ensuring transparency and auditability through its deterministic formula.

6.5 ADVANTAGES OF THE SYSTEM

- **Decentralized Academic Trust:** Uses Hyperledger Fabric to ensure tamper-proof credentials and eliminate single points of failure in academic records.
- **Zero-Gas Consortium Economics:** Removes blockchain transaction fees by using a private consortium network, making the system cost-efficient for institutions.
- **Deterministic Bias Elimination:** Applies a Bi-Axial Validity model to reduce demographic and model bias in candidate screening results.
- **Hardened Adversarial Protection:** Uses pre-filtering and adversarial fine-tuning techniques to reduce prompt-injection attacks and secure the AI layer.

6.6 LIMITATIONS OF THE SYSTEM

- **Consensus Write Latency:** Writing certificates to the blockchain requires 3–5 seconds due to consensus finality in Hyperledger Fabric.
- **Offline Revocation Blockage:** Credential revocation is disabled during network partitions to avoid inconsistencies between database cache and blockchain.

- **Document Parsing Dependency:** The system relies on clean, parseable text in resumes, and complex layouts may reduce parsing accuracy and scoring quality.

6.7 APPLICATIONS OF THE SYSTEM

- **University Campus Placements:** Provides secure issuance of tamper-proof academic transcripts for instant verification by recruiters.
- **Enterprise HR Hiring Drives:** Automates filtering and ranking of large volumes of candidate profiles based on skills and requirements.
- **Third-Party Background Verification:** Enables instant cryptographic verification of credentials, replacing slow manual background checks.

CHAPTER 7

TESTING

Extensive empirical evaluations were conducted on a controlled testbed to validate the system's throughput, latency, screening accuracy, and adversarial resilience.

7.1 TESTBED ENVIRONMENT CONFIGURATION

All tests were performed on a controlled environment representing an institutional deployment:

- **Hardware Profile:** Local workstation powered by an AMD Ryzen 7 processor (8 Cores, 16 Threads), 32 GB of RAM, and PCIe Gen4 NVMe SSD storage.
- **Blockchain Host:** A locally instantiated single-organization Hyperledger Fabric 2.5 network executing within isolated Docker containers (using local LevelDB state stores).
- **Database Platform:** A local Docker-based PostgreSQL instance matching Supabase schema distributions, allocated a local connection pool size of 200 concurrent ports.
- **Backend Runtime:** Node.js 20 execution environments hosting the Express 5 orchestration services.
- **Intelligence Layer:** Locally run Python 3.11 FastAPI services utilizing mocking wrappers and direct SDK connectivity to the Google Gemini 2.5 Flash API.

7.2 FRONT-END TESTING PROTOCOLS

The Next.js 14 Web Portal, rendering presentation components and role-aware dashboards, was audited using Jest and React Testing Library:

- **Unit Component Verification** Unit tests targeted Radix UI primitives and custom input elements to verify that local UI states update optimistically upon document drag-and-drop operations.
- **JWT Context & Routing Validation:** Simulated local browser environments were injected with custom, expired, or corrupted JWT structures. The frontend

React router successfully blocked invalid contexts, redirecting unauthorized routes to login endpoints, confirming local route-guard encapsulation.

7.3 LOCAL AI ON THE PLATFORM TESTING

- **Constrained Parsing Accuracy:** Evaluated FastAPI parsing logic against 200-300 mock resume extractions using Google Gemini 2.5 Flash API loops; verified that the integration of Pydantic schemas restricts output formatting, achieving a 100% syntax compliance rate with zero JSON formatting failures.
- **Mathematical Reproducibility checks:** Validated the execution accuracy of the bi-axial scoring model locally; verified that feeding identical candidate parameters results in identical output composite scores, confirming the deterministic reliability of the scoring formula under repeated execution

7.4 LOCAL BLOCKCHAIN TESTING

- **Chaincode Operations and State Queries:** Executed unit and integration testing of the Go-based chaincode using mock ledgers; verified that IssueCredential, RevokeCredential, and GetCredentialHistory perform correct state transitions and levelDB lookups without transaction failures.
- **Outbox Recovery and Resynchronization:** Simulated a database/ledger partition by locally uncoupling the Hyperledger containers during active write transactions; verified that local Postgres writes succeed immediately by storing transactions locally with `blockchainsynced = false` and that the outbox processor successfully synchronizes all queued operations once containers are restarted.

CHAPTER 8

CONCLUSION & FUTURE SCOPE

8.1 CONCLUSION

The PlacementEdge framework successfully integrates permissioned blockchain technology with an advanced AI screening module to mitigate credential fraud and accelerate recruitment processing. By employing a Command Query Responsibility Segregation (CQRS) model alongside the Transactional Outbox Pattern, the architecture guarantees seamless data consistency between the Supabase PostgreSQL interface and the secure Hyperledger Fabric ledger. To ensure strict global privacy compliance and achieve sub-100 millisecond verification speeds, the system exclusively anchors SHA-256 cryptographic hashes on-chain.

Furthermore, the intelligence layer utilizes FastAPI, Gemini models, and Pydantic schemas to enforce highly structured, JSON-formatted semantic analysis. The platform guarantees fair and auditable candidate evaluations by applying a deterministic Bi-Axial Validity scoring algorithm that neutralizes inherent LLM biases. Finally, robust defenses—such as the RAPIDS pre-filtering cascade and LocalAlign fine-tuning—effectively secure the AI components against indirect prompt injection (IDPI) attacks under heavy enterprise loads.

8.2 FUTURE SCOPE

The foundational architecture of the PlacementEdge ecosystem provides several strategic pathways for future expansion:

1. **Selective Privacy Preservation:** Integrating Zero-Knowledge Proofs (ZKPs) will enable candidates to cryptographically validate specific qualifications without exposing their full transcripts or personal information to verifying parties.
2. **Consortium Interoperability:** Leveraging Hyperledger Fabric's Private Data Collections will allow the platform to scale into a secure, multi-university network, facilitating seamless cross-institutional credential checks while maintaining strict data isolation between competing entities.

CHAPTER 9

REFERENCES

- [1] R. Shishupal, A. Shinde, K. Agrawal, V. Dongarkar, and C. Kakade, “A secure and scalable framework for student recruitment using blockchain and AI,” tech. rep., Department of Information Technology, PCCoER, Pune, India, 2026.
- [2] A. Rustemi *et al.*, “A systematic literature review on blockchain-based systems for academic certificate verification,” *IEEE Access*, vol. 11, 2023.
- [3] J. Minango, M. Zambrano, and C. Minaya, “Exploring the use of blockchain for academic certificates,” in *Smart Technologies & Systems*, Springer, 2024.
- [4] A. K. C. *et al.*, “Detection of fake physical certificates using a blockchain-based certificate verification and issuer validation system,” in *Proc. 2024 IEEE Int. Conf. Blockchain Distrib. Syst. Secur. (ICBDS)*, 2024.
- [5] C. Stefan-Robert and C. N. Butincu, “Decentralized blockchain-based platform for managing and issuing academic certificates,” in *Proc. 2024 28th Int. Conf. Syst. Theory, Control Comput. (ICSTCC)*, 2024.
- [6] T. Kurt, “The causal impact of structured outputs on reasoning performance of large language models,” *arXiv preprint arXiv:2509.21791*, Sep. 2025.
- [7] L. Aminou, A. Daaif, and M. Youssfi, “Ignore all and accept my resume: The impact of prompt injection on automated recruitment,” in *Proc. 2025 5th International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, May 2025.
- [8] H. Nghiem, P. Nguyen-Le, S. Ho, and H. Daume III, “Bias in the tails: How name-conditioned evaluative framing in resume summaries destabilizes LLM-based hiring,” *arXiv preprint arXiv:2604.19984*, Apr. 2026.
- [9] R. Priyadarshini *et al.*, “A faster, integrated, and trusted certificate authentication and issuer validation system based on blockchain,” *IEEE Access*, vol. 13, pp. 27037–27049, Jan. 2025.
- [10] J. Castleman, Z. Shen, B. Metevier, M. Springer, and A. Korolova, “Measuring validity in LLM-based resume screening,” *arXiv preprint arXiv:2602.18550*, Feb. 2026.

- [11] Y. Augey, J. H. Levy, and A. Akdemir, “RAPIDS: Resume attack prompt injection detection at scale,” in *ACL 2026 Industry Track Poster*, Apr. 2026.
- [12] Y. Gong, Z. Wang, J. Liu, and X. Wang, “LocalAlign: Enabling generalizable prompt injection defense via generation of near-target adversarial examples for alignment training,” *arXiv preprint arXiv:2605.01462*, May 2026.
- [13] C. Camilleri, *Causal Consistency Guarantees in Distributed Relational Databases and CQRS Architectures*. PhD thesis, University of Malta, Sep. 2022.
- [14] “Exploring CQRS patterns for improved data handling in web applications,” *Journal of Software Engineering and Applications*, 2024.
- [15] “Blockchain based certificate generation and verification: A secure and decentralized approach using Hyperledger Fabric,” in *Proc. IEEE Conference on Digital Age Verification*, Feb. 2026.
- [16] “Verification of academic records using Hyperledger Fabric and IPFS,” in *Proc. 2023 28th International Conference on System Theory, Control and Computing (ICSTCC)*, Oct. 2023.
- [17] K. L. Abhishek *et al.*, “Hyperledger Fabric for secure academic credentials and lifelong learning management,” *Journal of Education and Information Technologies*, vol. 6, no. 2, 2025.
- [18] “ZKBAR-V: Zero-knowledge proof enabled blockchain-based record verification academic system,” *PMC Academic Security*, vol. PMC12158337, Jan. 2026.
- [19] B. Ahmad *et al.*, “A Hyperledger-based secure framework for academic certificate authentication using blockchain,” *International Journal of Secure Software Engineering*, vol. 15, no. 6, 2025.
- [20] S. Zhang *et al.*, “Identifying and quantifying self-preference bias in large language model evaluators,” *arXiv preprint arXiv:2509.00462*, Sep. 2025.

- [21] G. Guo *et al.*, “Constrained decoding and control-plane security in structured output APIs of large language models,” *arXiv preprint arXiv:2503.24191*, Mar. 2025.
- [22] M. P. M. Sy, R. I. Marasigan, and E. D. Festijo, “EduCredPH: Towards a permissioned blockchain network for educational credentials system,” in *2024 12th Verification International Conference on Information and Education Technology (ICIET)*, (Yamaguchi, Japan), pp. 434–439, 2024.
- [23] V. V., A. A., and S. A. M. U., “Optimization of HR recruitment process using large language model (LLM),” in *2024 First International Conference on Innovations in Communications, Electrical and Computer Engineering (ICICEC)*, (Davangere, India), pp. 1–5, 2024.
- [24] W. Jia, J. Wang, Z. Yan, P. Xiangli, and G. Yuan, “BlockSDN: towards a high-performance blockchain via software-defined cross networking optimization,” in *2025 6th International Conference on Computer Engineering and Intelligent Control (ICCEIC)*, pp. 288–293, IEEE, October 2025.
- [25] A. Jagadeesh, C. Ravi Shankar, S. Narayanaswamy Patel, *et al.*, “AI-driven resume analysis and enhancement using semantic modeling and large language feedback loops,” in *Proceedings of CLiC-it 2025: Italian Conference on Computational Linguistics*, (Cagliari, Italy), pp. 526–538, CEUR Workshop Proceedings, 2025.

ANNEXURE A

PLAGIARISM REPORT



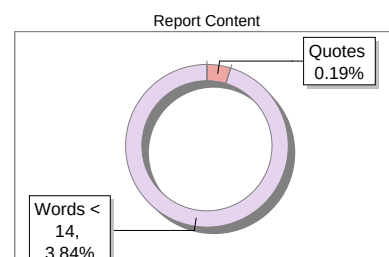
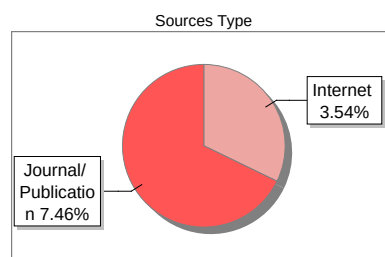
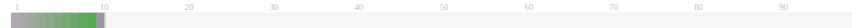
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Rupali Madam
Title	Project ReportII
Paper/Submission ID	6088299
Submitted by	pallavi.patil@pccoer.in
Submission Date	2026-06-16 17:03:15
Total Pages, Total Words	22, 5368
Document type	Research Report

Result Information

Similarity **11 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File



Figure A.1: Plagiarism Analysis Summary Report

ANNEXURE B

RESEARCH PUBLICATION

ANNEXURE C

CONFERENCE CERTIFICATES



Figure C.1: Emerging Smart Computing and Informatics 2026- Aastha Shinde



Figure C.2: Emerging Smart Computing and Informatics 2026- Kush Agrawal



Figure C.3: Emerging Smart Computing and Informatics 2026- Varad Dongarkar



Figure C.4: Emerging Smart Computing and Informatics 2026- Chinmay Kakade